Program 1: Working with Maven: Creating a Maven Project, Understanding the POM File, Dependency Management and Plugins

Step 1: To create a Maven project run the following command in terminal:

mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenApp -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false

This command generates a basic Maven project structure with a sample App.java and AppTest.java.

```
vboxuser@niranjan: ~/devopslab
vboxuser@niranjan:~$ cd devopslab/
vboxuser@niranjan:~/devopslab$ mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenApp -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
[INFO] Scanning for projects...
[INFO]
[INFO] ------------------< org.apache.maven:standalone-pom >-------------------
[INFO] Building Maven Stub Project (No POM) 1
[INFO] --------------------------------[ pom ]---------------------------------
[INFO]
[INFO] >>> maven-archetype-plugin:3.4.1:generate (default-cli) > generate-sources @ standalone-pom >>>
[INFO]
[INFO] <<< maven-archetype-plugin:3.4.1:generate (default-cli) < generate-sources @ standalone-pom <<<
[INFO]
[INFO]
[INFO] --- maven-archetype-plugin:3.4.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Batch mode
[INFO] No archetype defined. Using maven-archetype-quickstart (org.apache.maven.archetypes:maven-archetype-quickstart:1.0)
[INFO] ----------------------------------------------------------------------------
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-quickstart:1.0
[INFO] ----------------------------------------------------------------------------
[INFO] Parameter: basedir, Value: /home/vboxuser/devopslab
[INFO] Parameter: package, Value: com.example
[INFO] Parameter: groupId, Value: com.example
[INFO] Parameter: artifactId, Value: MyMavenApp
[INFO] Parameter: packageName, Value: com.example
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: /home/vboxuser/devopslab/MyMavenApp
[INFO] ----------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ----------------------------------------------------------------------------
[INFO] Total time:  5.091 s
[INFO] Finished at: 2026-03-24T16:27:09+05:30
[INFO] ----------------------------------------------------------------------------
vboxuser@niranjan:~/devopslab$
```

This “BUILD SUCCESS” will confirm that the project is created successfully

Step 2: Go to the project directory using command **cd MyMavenApp**

```
vboxuser@niranjan: ~/devopslab/MyMavenApp
vboxuser@niranjan:~/devopslab$ cd MyMavenApp/
vboxuser@niranjan:~/devopslab/MyMavenApp$
```

Step 3: Open and edit pom.xml file using command **gedit pom.xml**

```
vboxuser@niranjan: ~/devopslab/MyMavenApp
vboxuser@niranjan:~/devopslab/MyMavenApp$ gedit pom.xml
```

- Go to https://mvnrepository.com
- Search for a dependency “junit”
- Select the latest version and copy the dependency XML snippet.
- Paste it under the <dependencies> section in pom.xml
- Updated pom.xml:

Open pom.xml ~/devops/MyMavenApp Save

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">
 <modelVersion>4.0.0</modelVersion>
 <groupId>com.example</groupId>
 <artifactId>MyMavenApp</artifactId>
 <packaging>jar</packaging>
 <version>1.0-SNAPSHOT</version>


 <name>MyMavenApp</name>
 <url>http://maven.apache.org</url>

 <!-- Properties: Customize Java version or plugin versions -->
<properties>
<maven.compiler.source>11</maven.compiler.source>
<maven.compiler.target>11</maven.compiler.target>
</properties>
  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>3.8.1</version>
      <scope>test</scope>
    </dependency>
  </dependencies>

 <!-- Build: Configuring plugins and build settings -->
<build>
<plugins>
<!-- Example: Maven Compiler Plugin to compile Java code -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<version>3.8.1</version>
<configuration>
<source>1.7</source>
<target>1.7</target>
</configuration>
</plugin>

<!-- Example: Maven Surefire Plugin to run tests -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>2.22.2</version>
</plugin>


<plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-jar-plugin</artifactId>
            <version>3.1.0</version>
            <configuration>
                <archive>
```

XML Tab Width: 8 Ln 1, Col 1 INS

```xml
</plugin>

<!-- Example: Maven Surefire Plugin to run tests -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>2.22.2</version>
</plugin>


<plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-jar-plugin</artifactId>
            <version>3.1.0</version>
            <configuration>
                <archive>
                    <manifestEntries>
                        <Main-Class>com.example.MainClass</Main-Class>
                    </manifestEntries>
                </archive>
            </configuration>
        </plugin>

</plugins>
</build>
</project>
```

XML Tab Width: 8 Ln 1, Col 1 INS

Step 4: Build the project using command **<u>mvn clean install</u>**

```
vboxuser@niranjan: ~/devops/MyMavenApp
vboxuser@niranjan:~/devops/MyMavenApp$ mvn clean install
[INFO] Scanning for projects...
[INFO]
[INFO] -----------------------< com.example:MyMavenApp >-----------------------
[INFO] Building MyMavenApp 1.0-SNAPSHOT
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- maven-clean-plugin:2.5:clean (default-clean) @ MyMavenApp ---
[INFO] Deleting /home/vboxuser/devops/MyMavenApp/target
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ MyMavenApp ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/vboxuser/devops/MyMavenApp/src/main/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:compile (default-compile) @ MyMavenApp ---
[INFO] Changes detected - recompiling the module!
[WARNING] File encoding has not been set, using platform encoding UTF-8, i.e. build is platform dependent!
[INFO] Compiling 2 source files to /home/vboxuser/devops/MyMavenApp/target/classes
[INFO]
[INFO] --- maven-resources-plugin:2.6:testResources (default-testResources) @ MyMavenApp ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/vboxuser/devops/MyMavenApp/src/test/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:testCompile (default-testCompile) @ MyMavenApp ---
[INFO] Changes detected - recompiling the module!
[WARNING] File encoding has not been set, using platform encoding UTF-8, i.e. build is platform dependent!
[INFO] Compiling 1 source file to /home/vboxuser/devops/MyMavenApp/target/test-classes
[INFO]
[INFO] --- maven-surefire-plugin:2.22.2:test (default-test) @ MyMavenApp ---
[INFO]
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running com.example.AppTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.046 s - in com.example.AppTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- maven-jar-plugin:3.1.0:jar (default-jar) @ MyMavenApp ---
[INFO] Building jar: /home/vboxuser/devops/MyMavenApp/target/MyMavenApp-1.0-SNAPSHOT.jar
[INFO]
[INFO] --- maven-install-plugin:2.4:install (default-install) @ MyMavenApp ---
[INFO] Installing /home/vboxuser/devops/MyMavenApp/target/MyMavenApp-1.0-SNAPSHOT.jar to /home/vboxuser/.m2/repository/com/example/MyMavenApp/1.0-SNAPSHOT/MyMavenApp-1.0-SNAPSHOT.jar
[INFO] Installing /home/vboxuser/devops/MyMavenApp/pom.xml to /home/vboxuser/.m2/repository/com/example/MyMavenApp/1.0-SNAPSHOT/MyMavenApp-1.0-SNAPSHOT.pom
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  5.760 s
[INFO] Finished at: 2026-03-24T16:43:35+05:30
[INFO] ------------------------------------------------------------------------
vboxuser@niranjan:~/devops/MyMavenApp$
```

The “Build Success” message ensures that our Maven project is built successfully.

Step 5: To run the Maven application type he command:

<u>java -jar target/MyMavenApp-1.0-SNAPSHOT.jar</u>

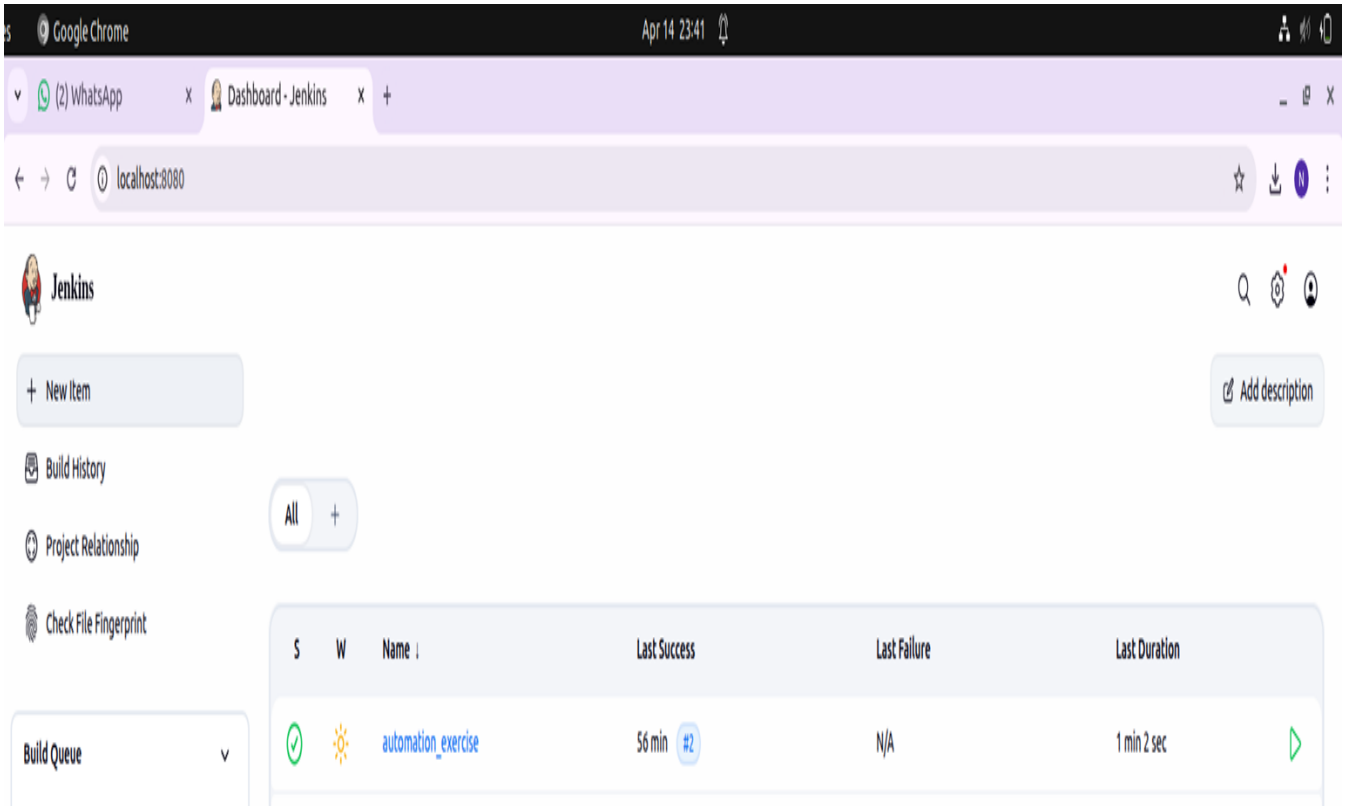
- After completing the execution, the output will look like:

```
vboxuser@niranjan: ~/devops/MyMavenApp
vboxuser@niranjan:~/devops/MyMavenApp$ java -jar target/MyMavenApp-1.0-SNAPSHOT.jar
Hello everyone
vboxuser@niranjan:~/devops/MyMavenApp$
```

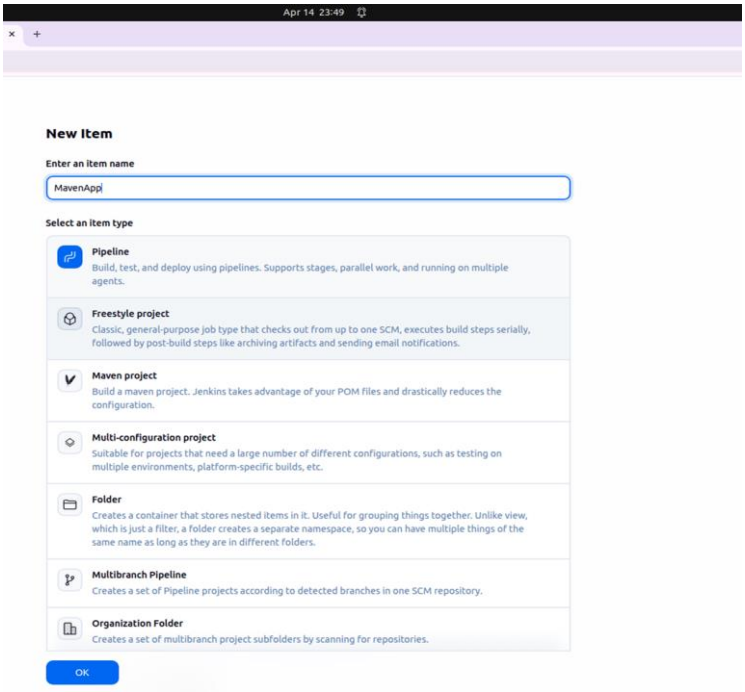
This will show output as the code present in App.java file we can edit the text to get any other output.

Jenkins Pipeline for MyMavenApp

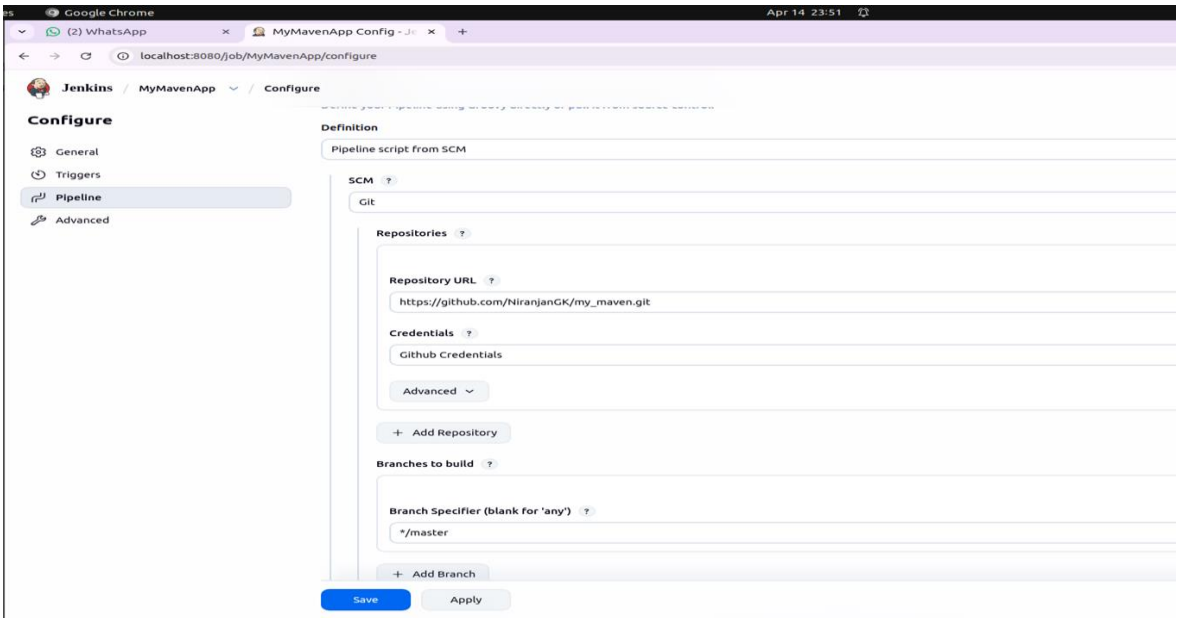
Step 1: Open the Jenkins Dashboard to create a new item



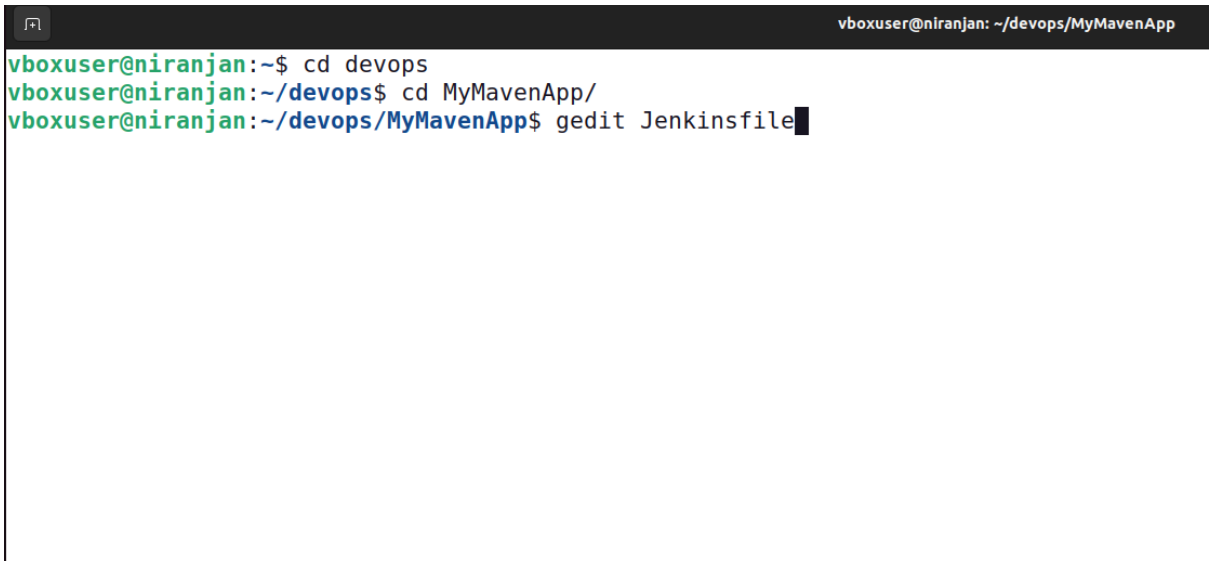
Step 2: Click on New Item and give the item name and select Item type as **Pipeline** and click on **OK**



Step 3: Configure the settings set script as “Pipeline Script from SCM” and SCM as “Git”.now paste the Repository URL of the project given in GitHub and choose Credentials as “GitHub Credentials” which was created earlier.



Step 4: Now in Terminal, go to directory created for MyMavenApp and create an new file called “Jenkinsfile”



Step 5: Paste the available code inside the Jenkinsfile

```
pipeline {
    agent any  // Use any available agent

    tools {
        maven 'Maven'  // Ensure this matches the name configured in Jenkins
    }
    stages {
        stage('Checkout') {
            steps {
                git branch: 'master', url: 'https://github.com/NiranjanGK/my_maven.git'
            }
        }

        stage('Build') {
            steps {
                sh 'mvn clean package'  // Run Maven build
            }
        }

        stage('Test') {
            steps {
                sh 'mvn test'  // Run unit tests
            }
        }




        stage('Run Application') {
```

```
        steps {
            // Start the JAR application
            sh 'java -jar target/MyMavenApp-1.0-SNAPSHOT.jar'
        }
    }



}


post {
    success {
        echo 'Build and deployment successful!'
    }
    failure {
        echo 'Build failed!'
    }
}
}
```

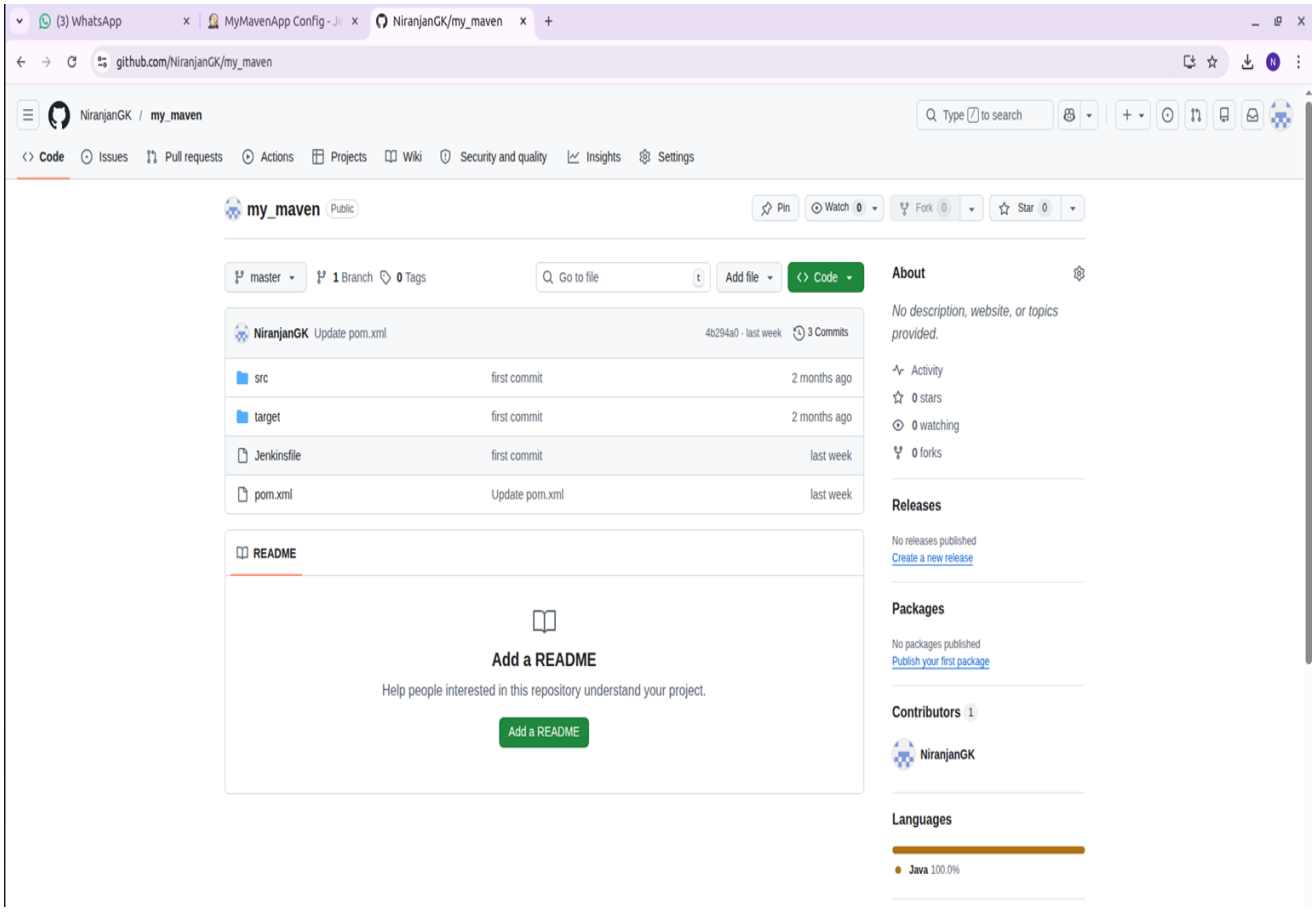
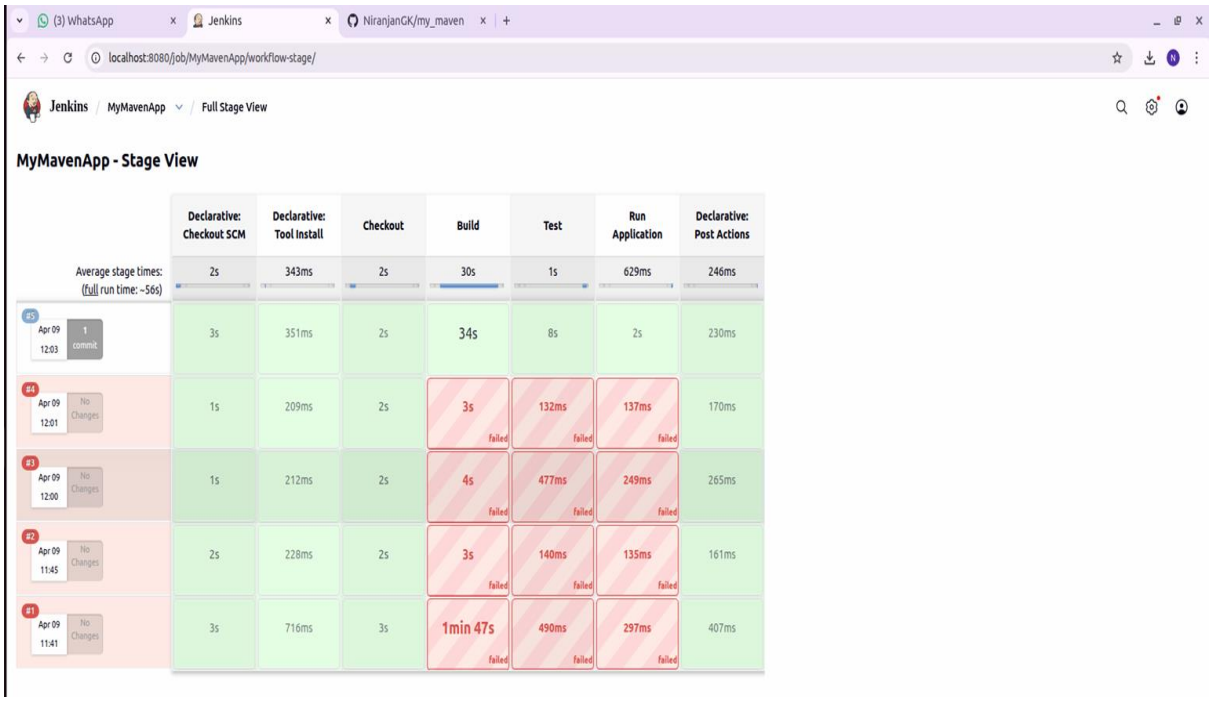
Open *Jenkinsfile ~/Devops/MyMavenApp Save

```
pipeline {
    agent any  // Use any available agent
    tools {
        maven 'Maven'  // Ensure this matches the name configured in Jenkins
    }
    stages {
        stage('Checkout') {
            steps {
                git branch: 'master', url: 'https://github.com/sumanthr27/MyMavenApp.git'
            }
        }
        stage('Build') {
            steps {
                sh 'mvn clean package'  // Run Maven build
            }
        }
        stage('Test') {
            steps {
                sh 'mvn test'  // Run unit tests
            }
        }
        stage('Run Application') {
            steps {
                // Start the JAR application
                sh 'java -jar target/MyMavenApp-1.0-SNAPSHOT.jar'
            }
        }
    }
    post {
        success {
            echo 'Build and deployment successful!'
        }
        failure {
            echo 'Build failed!'
        }
    }
}
```

Step 6: make sure that in pom.xml the versions of sorce and target is correct

```
<!-- Example: Maven Compiler Plugin to compile Java code -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<version>3.8.1</version>
<configuration>
<source>1.8</source>
<target>1.8</target>
</configuration>
</plugin>
```

Step 7: push the Jenkinsfile that is created to Github.



Step 8: Now go to Jenkins and Build the Pipeline and check the status in Stage view.